STM JOURNALS

Journal of VLSI Design Tools & Technology
ISSN: 2249-474X (Online), ISSN: 2321-6492 (Print)
Volume 8, Issue 1
www.stmjournals.com

AXI Bridge and DMA/Bridge Subsystem for PCIe

Shivani Malhotra[1,*], Neelam Rup Prakash[2]
[1]VLSI Design Laboratory, Department of Electronics and Communication Engineering, Punjab Engineering College (Deemed to be University), Chandigarh, Punjab, India
[2]Department of Electronics and Communication, PEC University of Technology, Chandigarh, India

Abstract

We know that the present-day SOC design consists of many intellectual property cores with several communication protocols. To enhance the performance of the whole system, we need to improve the bridging and signal translation between these protocols. In this article, discussion has been carried out on PCIe and AXI protocol from AMBA. PCI Express, because of its various features like high speed, low power consumption, and good protocol efficiency can be thought of as a quality prospect for interconnects. On the other hand, AXI protocol from AMBA offers on chip communication for designing high performance systems. Combination of this on-chip and off-chip interfaces can yield revolutionary results. In this paper, a bridge core design has been proposed which can work in two modes which are AXI Bridge and DMA subsystem for PCIe.

Keywords: *PCIe (Peripheral Component Interconnect Express), AXI (Advanced Extensible Interface), DMA (Direct Memory Access), AGP (Accelerated Graphics Port)*

****Author for Correspondence*** E-mail: shivani.malhotra610@gmail.com

INTRODUCTION

Introduction to PCIe

Due to advancements in silicon technologies, data acquisition rates have also increased. As a result, large data chunks must be sent to the system for processing. A data bus which links the device with the memory, handles these transfers. So, that is when PCI and AGP came into being.

PCI abbreviated as Peripheral Component Interconnect is a native bus which is used for connecting peripheral devices to a computer. In another way, we can say that it performs "off-chip" communication. But with the further advent in technology, PCI had to face bandwidth limitations, issues in real time data transfers etc. So to eliminate these limitations, another bus technology called PCIe came into being [1].

PCIe abbreviated as PCI express has successfully established itself as a successor to PCI and AGP. It is basically a bus standard for connection of devices to computer. PCIe actually refers to expansion slots on motherboard. Some salient features of PCIe which make it superior to PCI and AGP are as follows:

1) As already mentioned, PCIe fulfils high bandwidth requirement in between the device and the motherboard, which was a big limitation in PCI. The use of graphics cards with PCIe has been greatly benefited from this huge bandwidth.
2) This version of PCI that is PCI express is an improved version in terms of speed. The difference in speed is quite large as PCI slot runs at 133 Mb/s to a 16 slot lane whereas PCIe runs at a speed of 16 Gb/s to a 16 slot lane, thereby providing a speed of 1 Gb/s to one lane.
3) PCIe uses a serial lane instead of a parallel lane as was used by PCI, but in contrast, PCIe has employed individual buses for its tasks instead of a shared bus.
4) The access to the older PCI is arbitrated and restricted to a single master at one time and only in one direction as it has a shared bus topology. All peripheral devices are connected to the root complex by individual links. But on the other hand, the improved version that is PCIe employs full duplex communication which is concurrent over multiple endpoints.

COMPONENTS OF PCIe

Now we will discuss about the components of PCIe. In the PCIe architecture, there are many components we need to understand about. These components and their interconnections will help us understand about the off chip communication carried out from PC to peripheral hardware devices. The components are as follows (Figure 1):

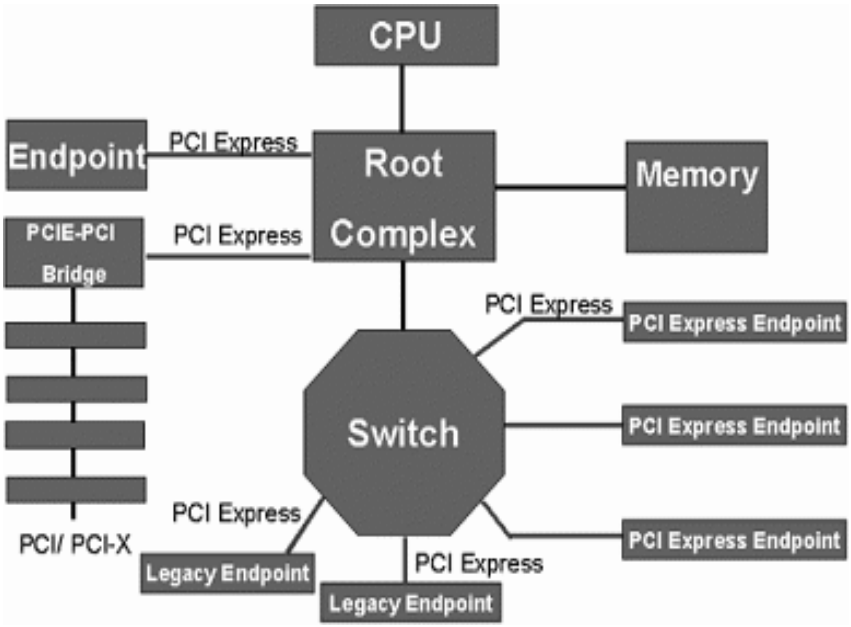

Fig. 1: *PCIe Architecture.*

Root Complex
It links CPU to PCIe topology. Transactions are generated by root complex on behalf of CPU. Data can flow from CPU to an end point. It connects host CPU/memory complex to PCI Express hierarchy. It is not limited to a single device and can have one or more downstream ports.

End-Point
It forms the lower part of the PCIe Tree topology. It has only one upstream port and is responsible for requesting or completing transactions.

Legacy End-Point
It utilizes old PCI bus operations to become backward compatible.

Switch
Peer-to-peer support is provided by switches which perform as packet routers across the ports which is mandatory to be done. It has the following features:

1. One upstream port directed towards root complex.
2. It can have one or more downstream ports.
3. They can be stacked.
4. Peer to peer traffic is allowed.

Bridge
It connects different buses. One upstream port is directed towards root complex and one downstream port to other devices. Example: PCI or PCI-X bus (In forward it is from PCIe to PCI and in reverse, it is from PCI to PCIe).

Here we need to know that there is no concept of master and slave in PCIe. Since the interface is point to point, we only have a requester and a completer of the transaction.

PCIe ARCHITECTURE
PCI express has a layered architecture that has the ability to adapt to new technologies. These layers are responsible for the data transfers between two devices, deciding the signaling rates etc. The layers are as follows [2]:

- Physical Layer,
- Data Link Layer, and
- Transaction Layer.

The layered architecture of PCIe is displayed in Figure 2 above. The layers of the PCI express are as follows [3]:

Layers of PCIe

Transaction Layer
In this layer, requests are transformed from device core to an authentic PCIe transaction. This layer forms the top part of the PCI Express design. Its main function is to receive, buffer, and transmit the transaction layer packets (TLPs). TLPs disseminate information by using memory, IO, configuration, and message transactions.

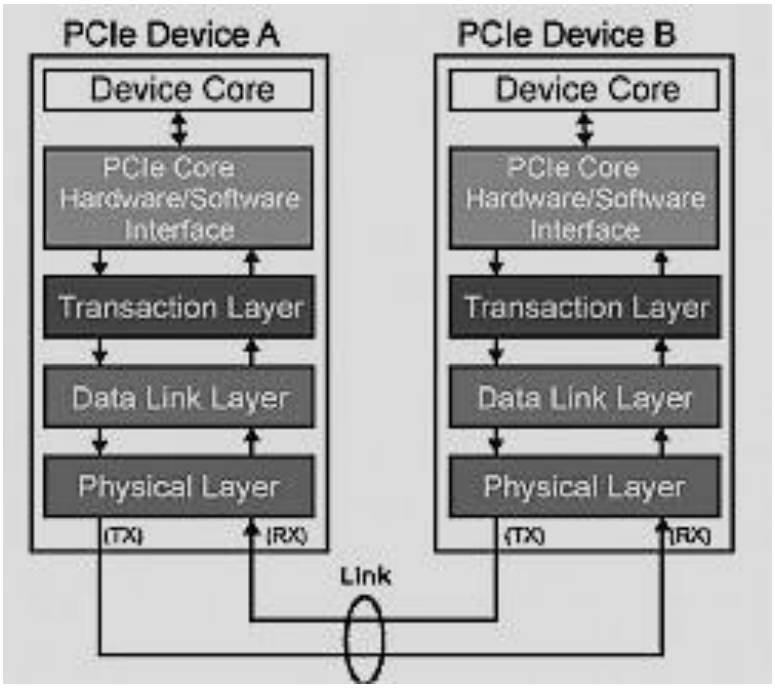

Fig. 2: *Layers of PCie Showing Data Transfer between Two Devices.*

Data Link Layer

This layer maintains the integrity of transactions across the link. It traces the status of the link and communicates it on to the other two layers. It is sandwiched between the transaction layer and the physical layer. Providing a responsible mechanism for the interchange of TLPs between two components on a link is the key responsibility of this layer.

Physical Layer

Physical layer acts as a transceiver across a PCIe link. When switched on, it begins with the number of layers to be used, link speed etc. Physical layer takes care of how the data should be transmitted because the other two layers, that is, TL and DLL are unaware of this task. The physical layer is responsible for framing, de-framing, scrambling, descrambling, 8b/10b encoding and decoding of TLPs and DLLPs.

Packet Translation across the Layers

Transmitter Side

Here the transaction layer (TL) gets the instruction from the PCIe core software layer and produces a TLP (transaction layer packet) which is then moved to the data link layer. Then the data link layer (DLL) attaches some more information to the received packet , checks its validity and then transfers the updated TLP now called DLLP (data link layer packet) to the physical layer. At this stage, the DLLP gets converted to an electrical format that can be transmitted across the link. Now the PL attaches frames to detect the start/end of packet and DLLP gets converted to PLP physical layer packet. This packet then gets transmitted across the link to the component on the receiver side [4].

Receiver Side

Here the reverse process occurs. The additional information added by its analogue layer on the transmit side is detached from each layer. Accuracy of information is confirmed and the packet is passed to the layer above with correct status.

Now to support today's high volume of data, SoC designers of high-performance computing and networking applications must leverage a scalable chip-to-chip interface that enables high data throughput while minimizing the latency and efficient management of power. PCIe was first known as a board level bus system in personal computers, but today, with its wider links, distributed computing capabilities, and higher data rates, PCIe enables external connectivity in SoCs for high-performance servers. So for this reason, we need to learn about leveraging PCI Express to

enable external connectivity in Arm-Based SoCs. For that purpose, ARM had introduced Advanced Microcontroller Bus Architecture in 1996.

ADVANCED MICROCONTROLLER BUS ARCHITECTURE (AMBA)

The ARM Advanced Microcontroller Bus Architecture (AMBA) contains interconnect specifications which help us to connect and manage the practically working blocks in system-on-a-chip (SoC) designs. AMBA specifications include many protocols/buses/interfaces, one of which is AXI4 that is Advanced Extensible Interface which comes under AMBA. This is one of the latest protocols which targets high performance, high clock frequency system designs and involves qualities due to which it is preferred for high speed sub-micron interconnects.

We have chosen this particular protocol because of its following features which should come in compatibility with the PCI express:

- It has individual buses for address/control and data phases.
- It supports unaligned data transfers by using byte strobes.
- It supports burst based transactions by issuing only the start address.
- It issues multiple outstanding addresses with out of order responses.
- We can easily add register stages for providing timing closure.

External Connectivity in Arm-Based SoCs

We cannot deny the fact that AXI and PCI Express have major key concepts in common; still it is a challenging task to build an efficient design of bridge between these two. We just cannot straight away map everything that they both share. Some important points that require a careful design to get controlled have been explained below:

Transfer Efficiency

The AXI protocol can carry the transmission of parallel read and write requests, and read completion in the same direction but that is not the case with PCI Express. It can carry out transmission of one single type of packet at one time. It should be noted that the choice of the packet for transmission affects the net performance of the system.

Also, both these protocols have different rates of transmission and reception of data packets. The PCI Express port has the ability to receive only one packet at a time but the AXI destination might not be able to welcome the received packet at the same moment.

Translating from AMBA Memory Transaction Ordering Model into/from PCIe Ordering Model

Traffic outbound from the SoC runs through AXI slave transactions. Similarly, traffic inbound to the SoC from PCIe will run through the AXI master transactions. A PCIe AXI Master needs to be compliant with the same ordering rules, so it must have very similar ordering logic as described above. Some paths can be simpler, for example, the inbound read path does not require ordering logic as long as it does not reorder inbound reads, since a compliant AXI slave ensures read-after-read, by ordering the read data completions. To ensure compliance with the read-after-write rule, the Master logic could simply wait for the write response before issuing the read.

For the PCIe slave interface to meet the Arm ordering model, it must properly handle:

- *Read-after-Read:* Since PCIe does not guarantee ordering between reads, this function must be handled by the ordering logic in the slave.
- *Write-after-Write:* It can generally be achieved by mapping to PCIe non-relaxed posted transactions except for PCIe configuration writes where ordering is not guaranteed.
- *Read-after-Write:* It is guaranteed by PCIe ordering rules. It ensures producer/consumer model with some options for relaxation if performance in the target application does require it and data is known to be unrelated.
- *Write-after-Read:* PCIe allows writes to pass reads to avoid deadlock scenarios. However, the CPU mainly controls this function, so applications needing to comply with this rule naturally do so.

Clock Domain Crossing

Inside the structure of bridge, it can occur that the PCI Express and AXI controllers would be running at their own distinct clock rates. For example, PCIe IP generally runs on a clock source derived from the PCIe data rate, so frequencies of 125, 250, 500, or 1000 MHz are common depending on PCIe signaling rate (2.5, 5, 8, or 16 GT/s), PHY interface width and the number of lanes being implemented. The internal clock of SoC may run at a different rate than the PCIe interface, such as 400, 800, or 1600 MHz and change drastically with application load. A well-designed PCIe interface IP will free the SoC designer from needing to consider the actual clock speed, link power state and transaction buffering to ensure gap-free transmission/reception on both the PCIe and SoC interfaces.

So, we can say that building an Arm-based SoC with a PCI Express interface requires deep knowledge of the PCIe protocol, the Arm AMBA protocol, ordering issues, different clocking domains, error mapping, tag management, etc., resulting in longer SoC development time, which designers can overcome with the use of 3rd-party PCIe IP. Designers can enable external connectivity in Arm-based SoCs and reduce their time to market by using a compliant PCIe IP that is proven in millions of devices, allowing designers to focus their attention on the rest of their SoC design. Integration of a proven PCIe IP helps overcome design challenges which have been discussed earlier.

BRIDGE IP TO ADD PCI EXPRESS IN SoCs

The development of a bridge between two standard protocols is an interesting building block for system designers. One such bridge design being reviewed and executed in bulk today is the bridge between PCI Express and AXI protocols. It has been found out by the PCI Express application designers that this bridge provides easy execution of its application on any AMBA-based SOC. Moreover, SOC designers are utilizing these bridges to talk to the PCI Express consistently which enables easy interface with I/O protocols.

Now talking about the bridge core, it integrates the two standard protocols which we have been discussing about, that is, PCIe and AXI. It consists of the memory mapped AXI4 to AXI4-Stream Bridge and the AXI4-Stream Enhanced Interface Block for PCIe. The former bridge consists of a register block and two functional half bridges, known as the slave and master bridge. All the interconnections can be understood from the Figure 3 given below [5].

Due to various transaction types of PCI Express, some special side band signals are required by the bridge (PCIe to AXI bridge) for providing information to report the condition of a particular transaction. With the side band signals, error mapping can be defined between errors of PCI Express (UR, CA, CRS, poisoned, and ECRC error) and errors of AXI Slave response (SLVERR and DECERR).

By providing AMBA 3 AXI bridge a PCI Express, designers using the AMBA 3 AXI protocol can be enabled to easily add PCI Express external network to their system on chips. This yields higher throughput designs with a joint to a lot of accessible systems and peripheral devices which are based on PCI Express technology [6].

Another consideration for SoC designers is where to place their DMA (Direct Memory Access) engine(s). While it is possible to use an off-the-shelf DMA engine communicating solely over the AMBA interconnect, there are limitations to such an architecture. To get the maximum performance, the DMA engine needs to understand both AMBA and PCIe [7].

The DMA/Bridge Subsystem for PCI Express can be configured to be either a high performance direct memory access (DMA) data mover or a bridge between the PCI Express and AXI memory spaces (Figure 4).

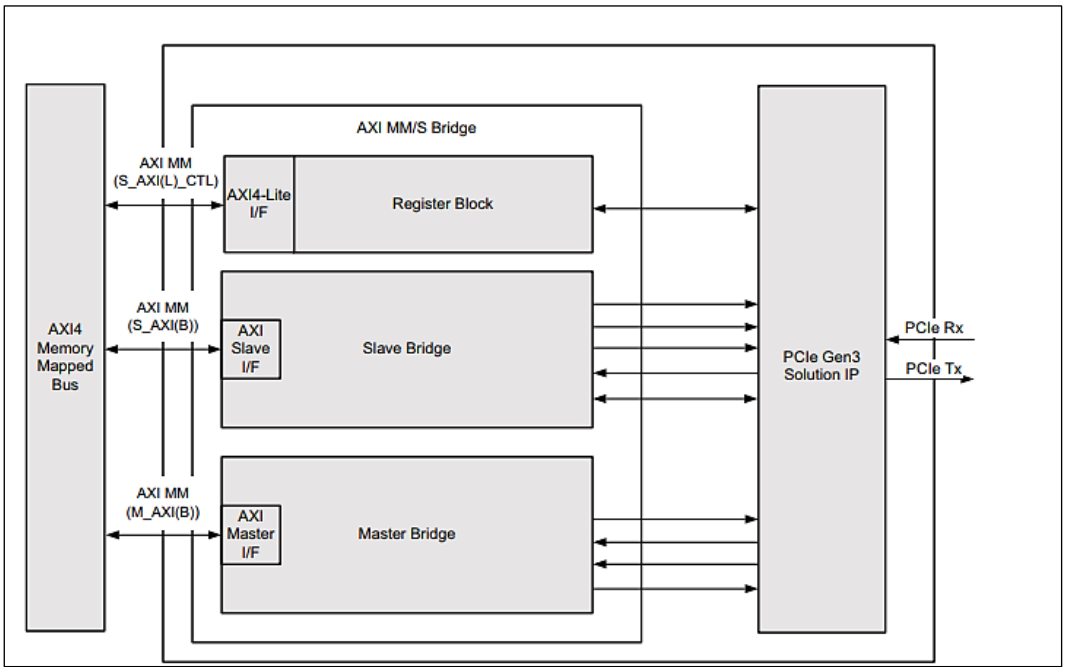

Fig. 3: *High-Level Bridge Architecture for the PCI Express Gen3 Architecture.*

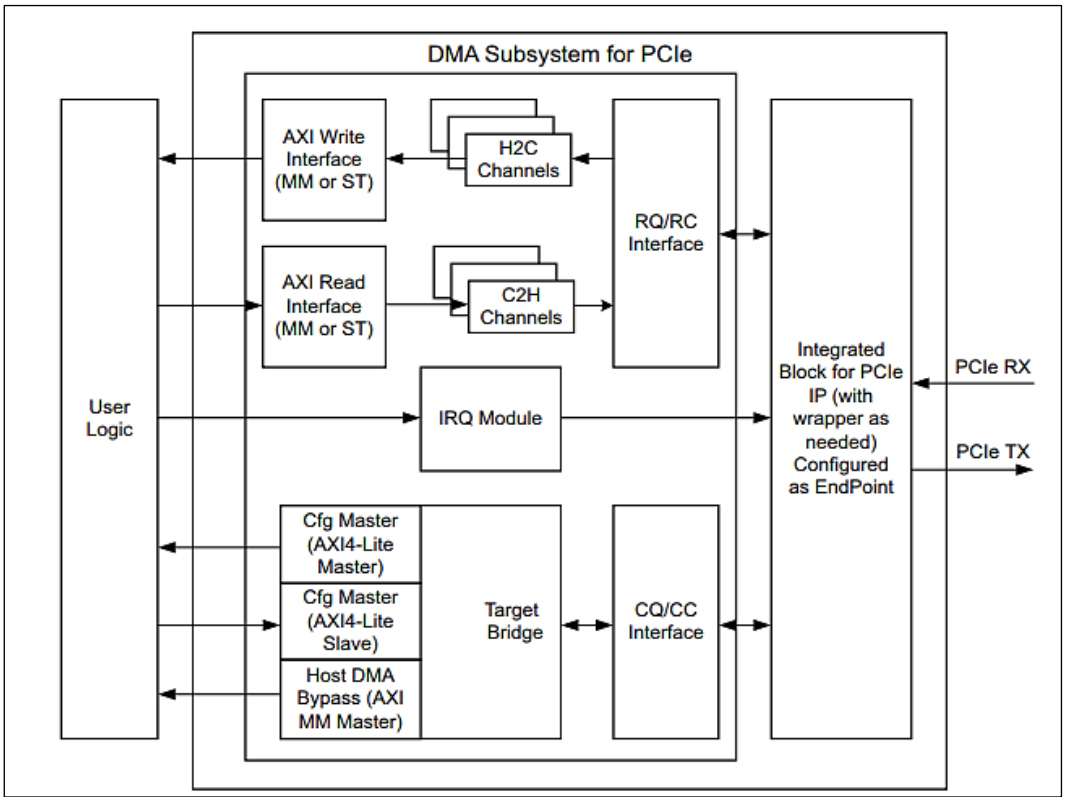

Fig. 4: *DMA Subsystem for PCIe Overview.*

- DMA Data Mover: As a DMA, the core can be configured with either an AXI (memory mapped) interface or with an AXI streaming interface to allow for direction connection to RTL logic. Either interface can be used for high performance block data movement between the PCIe address space and the AXI address space using the provided character driver. In addition to the basic DMA functionality, the DMA supports up to four upstream and downstream channels, the ability for

PCIe traffic to bypass the DMA engine (Host DMA Bypass), and an optional descriptor bypass to manage descriptors from the FPGA fabric for applications that demand the highest performance and lowest latency [5–9].

- Bridge between PCIe and AXI Memory: When configured as a PCIe Bridge, received PCIe packets are converted to AXI traffic and received AXI traffic is converted to PCIe traffic. The bridge functionality is ideal for AXI peripherals needing a quick and easy way to access a PCI Express subsystem. The bridge functionality can be used as either an Endpoint or as a Root Port [1].

Practical Significance

A broad range of computing and communications, focus on practical significance, prioritizing the presentation, market price, flexibility and mission-critical infallibility is ensured by the core architecture [8]. Typical applications include:

- Data communications networks.
- Telecommunications networks.
- Broadband wired and wireless applications.
- Network interface cards.
- Chip-to-chip and backplane interface cards.

Unsupported Features

The following are not supported by this core:

- Tandem Configuration solutions (Tandem PROM, Tandem PCIe, Tandem with Field Updates, PR over PCIe) are not supported for Virtex-7® XT and 7 series Gen 2 devices.
- Narrow burst.
- BAR translation for DMA addresses to AXI4 Memory Mapped interface.

CONCLUSION

To wrap-up, it can be said that PCI Express to AXI bridge module boosts the PCI Express implementation proposal and attaches appreciable value to a SOC based design. While designing a bridge like this is a troublesome job requiring substantial endeavor and proficiency, taking a third-party bridge solution can eventually minimize the time, market price and effort sustained by SOC designers.

REFERENCES

1. Ray Bittner. Speedy Bus Mastering PCI Express. *22nd International Conference on Field Programmable Logic and Applications*. Aug 2012.
2. PCI Express Base Specification, PCI SIG. Available: http://www.pcisig.com/specifications/pciexpress.
3. Alex Goldhammer, John Ayer Jr. Understanding Performance of PCI Express Systems. *Xilinx WP350*. Sep 4, 2008.
4. Ye Ren, Young Woo Kim, Hag Young Kim. Implementation of System Interconnection Devices Using PCI Express. *IEEE International Conference on Consumer Electronics (ICCE)*. 2015.
5. https://www.xilinx.com/support/documentation/ip_documentation/axi_pcie3/v3_0/pg194-axi-bridge-pcie-gen3.pdf
6. Zdravko Panjkov, Juergen Haas, Martin Aigner, *et al.* OCP2XI Bridge: An OCP to AXI Protocol Bridge. ARC 2014, LNCS 8405; 2014; 179–190p.
7. https://www.xilinx.com/support/documentation/ip_documentation/xdma/v4_0/pg195-pcie-dma.pdf
8. https://community.arm.com/soc/b/blog/posts/leveraging-pci-express-to-enable-external-connectivity-in-arm-based-socs
9. Qiang Wu, Jiamou Xu, Xuwen Li. The Research and Implementation of Interfacing Based on PCI Express. *Electronic Measurement & Instruments, 2009. ICEMI'09. 9th International Conference on.* 16–19 Aug 2009.

Cite this Article

Shivani Malhotra, Neelam Rup Prakash. AXI Bridge and DMA/Bridge Subsystem for PCIe. *Journal of VLSI Design Tools & Technology*. 2018; 8(1): 23–29p.

